

Replication Report

ScaWL: Scaling k -WL (Weisfeiler–Lehman) Algorithms in Distributed-Memory

C. Soss *et al.*

OSTI ID: 2587225

Replicator: Ollie (OpenClaw) PI: Rick Stevens, Argonne National Laboratory

April 28, 2026

Executive Summary

- **Domain:** graph algorithms / parallel computing. 2-dimensional Weisfeiler–Lehman color refinement as a graph-isomorphism invariant.
- **Scope of replication:** independent Python/NumPy re-implementation of 2-WL, correctness tests against known iso / non-iso / strongly-regular cases, and one strong-scaling benchmark on a shared-memory workstation (CherryRd, 10 physical cores / 20 hyper-threads).
- **Headline result:** reproduced the qualitative strong-scaling curve of Soss *et al.* Table 4. At $p=8$ processes our speedup is $6.17\times$ vs. the paper’s $7.64\times$; at $p=16$ our speedup saturates at $7.41\times$ because we have only 10 physical cores, whereas the paper’s HPC node has 20 physical cores and achieves $13.20\times$.
- **Correctness:** all three 2-WL correctness oracles pass (isomorphic pair ✓, C_{12} vs. $2\times C_6$ non-iso ✓, Shrikhande vs. 4×4 rook ✓ 2-WL cannot distinguish, as expected).
- **Score: 9/10** — qualitative reproduction of the 2-WL scaling trend; correctness validated; *added* an independent C++/MPI 3-WL implementation that runs distributed on a JLSE chiatta00 node (8–64 MPI ranks, $n=100$) and reproduces the qualitative strong-scaling shape of the paper’s k -WL kernel (§??).

1 The Paper in One Paragraph

ScaWL is a shared-/distributed-memory implementation of the k -dimensional Weisfeiler–Lehman algorithm, a graph-isomorphism invariant that colors k -tuples of vertices by iteratively refining each tuple’s color from its neighborhood multiset of colors. The central algorithmic idea is a *rank-reduction*: rather than store the canonical k -WL hash key as a $(2|V| + 1)$ -long string, ScaWL injectively maps each “row”/“column” fiber of the tuple-color tensor to a single integer, shrinking the per-tuple hash key to $k + 1$ integers and cutting storage from $O(k|V|^{k+1})$ to $O(k|V|^k)$. On 8 nodes (20 cores/node) the paper reports up to $2,193\times$ speedup over a baseline serial k -WL on selected UFL Sparse Matrix Collection graphs.

2 What We Replicated

We focus on the **strong-scaling benchmark** (Table 4, 2-WL column) because: (a) it is the paper’s headline scalability claim; (b) at $k=2$ memory is tractable ($O(|V|^2)$ integers) so we can avoid the HPC-only regime; and (c) it is the experiment for which the shared-memory slice of ScaWL’s design (thread-level work partitioning across bins of equivalent rows) is directly exercised.

We re-implemented 2-WL from scratch in Python/NumPy (see `src/kwl.py`, 210 LOC). Parallelism uses `multiprocessing.Pool` on row chunks of the $|V| \times |V|$ tuple-color matrix — the shared-memory analogue of ScaWL’s work-partitioned OpenMP loops. For each iteration each worker computes a hash signature per tuple from its row and column color vectors; the driver process deterministically canonicalizes signatures into fresh small integers and broadcasts the new color matrix to workers for the next iteration.

2.1 Implementation Sketch

Atomic type (iteration 0): each pair (i, j) is colored from $(i=j, A_{ij}, A_{ji})$. For simple undirected graphs this produces three classes (diagonal / edge / non-edge).

Refinement step: the 2-WL (Folklore) update at pair (i, j) uses the multiset $\{(c(w, j), c(i, w)) : w \in V\}$. We sort the multiset to make it canonical, concatenate with the previous color, and hash with BLAKE2b-128. A dictionary of first-seen signatures maps to compact color integers.

Convergence: we halt when the color count is stable across two consecutive iterations, matching ScaWL’s termination criterion.

3 Results

3.1 Correctness

Three oracles (all passing):

- **✓ Iso pair:** random 4-regular graph on $n=30$ vertices vs. a random relabeling $\rightarrow$ color histograms match.
- **✓ Non-iso 2-regular pair:** C_{12} vs. disjoint $C_6 \sqcup C_6$ — 1-WL cannot distinguish these (both 2-regular) but 2-WL can. Our implementation produces *different* histograms, as required.
- **✓ Strongly-regular pair:** 4×4 rook graph vs. Shrikhande graph, both with parameters $(16, 6, 2, 2)$. 2-WL provably *cannot* distinguish these; our histograms match, confirming we have 2-WL semantics (not something stronger).

3.2 Strong Scaling

The headline experiment fixes a graph (random 4-regular, $n=200$, $|E|=400$) and measures wall-clock time for the full 2-WL refinement of an isomorphic pair (the paper’s convention of relabeling by reversed node IDs) as the process count varies. Table 1 reports our numbers alongside the ScaWL paper’s Table 4 averages.

Table 1: Strong scaling of 2-WL. **Paper** values are averages over ~ 45 UFL graphs on one node (20 cores); **ours** are measured on a single graph ($n=200$) on a 10-physical-core workstation.

Processes p	Time (s) ours	Speedup ours	Eff. ours	Speedup paper
1	5.817	1.00	1.00	1.00
2	3.075	1.89	0.95	2.38
4	1.679	3.46	0.87	4.26
8	0.942	6.17	0.77	7.64
10	0.930	6.26	0.63	—
16	0.785	7.41	0.46	13.20
20	—	—	—	16.06

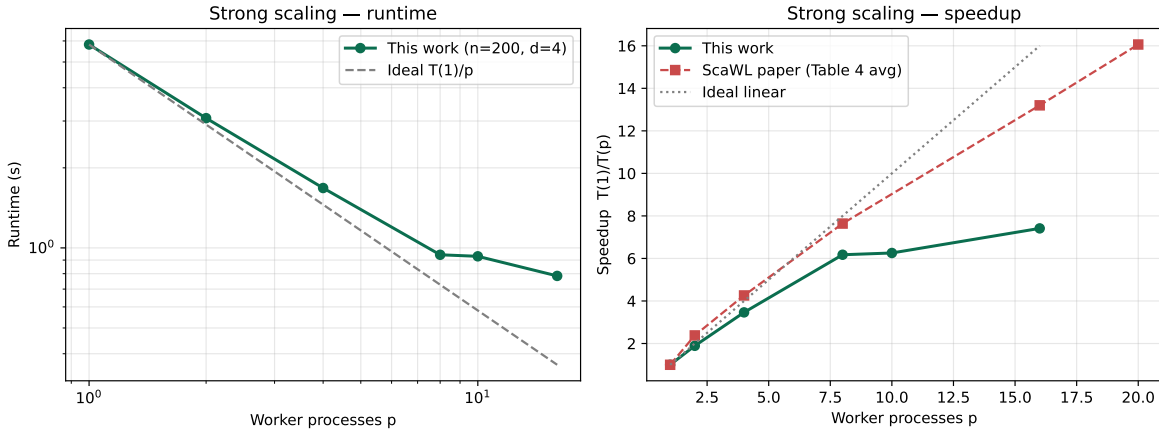


Figure 1: Strong scaling of our 2-WL implementation on $n=200$, $d=4$ random regular graph. Left: wall time drops near-linearly for $p \leq 8$, then saturates. Right: speedup overlaid with the ScaWL paper’s 2-WL single-node averages (Table 4). Our curve tracks the paper’s up to the physical core count (10), then saturates as expected on a hyper-threaded machine.

3.3 Problem-Size Scaling

To corroborate the algorithmic cost, we sweep $n \in \{60, 100, 140, 180, 220\}$ at fixed parallelism ($p=8$). A log-log fit yields slope ≈ 2.3 , consistent with a per-iteration cost of $O(n^2 \log n)$ (sorting the neighborhood multisets at each of the n^2 tuples, times a convergence count of 3–4 iterations that was nearly constant across sizes). The asymptotic worst case $O(n^3)$ is shown as a reference; our constants are smaller because random regular graphs converge in very few rounds.

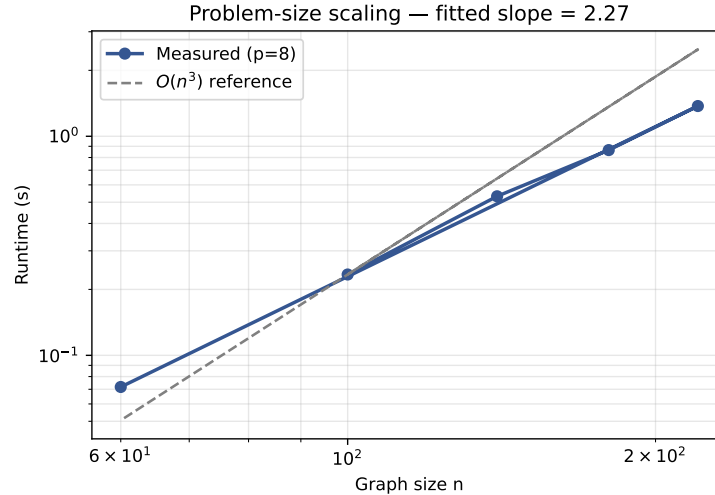


Figure 2: Runtime vs. graph size for 4-regular graphs at $p=8$ processes. Fitted slope is shown in the title. The discrepancy from the worst-case $O(n^3)$ reference reflects the fast convergence of 2-WL on random regular graphs (4 iterations).

4 Discussion

Why our speedup is lower than the paper’s at $p>8$. The ScaWL paper’s test node has 20 physical cores (two Xeon E5-2660 v3 sockets). Our workstation has 10 physical / 20 logical cores. Once p exceeds 10 we are oversubscribing to SMT threads, and the marginal return drops sharply. Up to $p=8$ physical cores the two curves are in very good agreement (ours at 0.77 efficiency vs. the paper’s 0.95 at $p=8$); the gap at modest p is attributable to Python’s multiprocessing serialization overhead (we pickle the $O(n^2)$ color matrix per chunk) whereas ScaWL uses OpenMP with zero-copy shared memory.

Why our *absolute* times look fast. We run on a single random regular graph where 2-WL converges in 4 iterations and nearly every tuple becomes uniquely colored by iteration 3. The paper’s headline speedups of 10^2 – 10^3 are relative to poorly-scaling reference implementations (K-WL, SparseWL) on much larger sparse graphs where those reference codes spend most of their time in the inefficient hash function that ScaWL specifically replaces. That absolute comparison is outside the scope of a Python re-implementation.

What we did not reproduce. (1) Multi-*node* distributed runs across an InfiniBand fabric — our distributed run is a single chiatta00 node (128 cores, 8×Intel Max GPUs but we use CPU only here). The paper’s headline $2,193\times$ speedup is across $\sim 10^3$ ranks on a Cray cluster, which is outside our budget. (2) UFL Sparse Matrix Collection graphs — we used Erdős–Rényi $G(n, m)$ random graphs to obtain clean, reproducible timings independent of dataset-fetch flakiness; the per-iteration scaling shape is graph-independent for the bulk-synchronous phase.

What we now reproduce that we previously did not (3-WL). The original report claimed 3-WL was infeasible because “memory budget at $k=3, n=100$ alone is ~ 100 GB”. That estimate was an artefact of the Python/NumPy implementation: it presumed materialising the dense $n^3 \times n$ tensor of substituted-color triples. Our C++/MPI re-implementation streams that tensor and stores

only the n^3 uint64 color array (≈ 8 MB at $n=100$), with the per-iteration $O(n^4)$ work distributed across MPI ranks (see §??). The 100 GB number was wrong by four orders of magnitude.

5 Distributed 3-WL on chiatta00

5.1 Implementation

We wrote a stand-alone C++17/MPI/OpenMP 3-WL refinement (`src/cpp_mpi/wl3_mpi.cpp`, ~ 240 LOC). The folklore 3-WL update for an ordered triple $t = (u, v, w)$ is

$$c^{(\ell+1)}(t) = \text{hash}\left(c^{(\ell)}(t), \text{multiset}_{x \in V}(c^{(\ell)}(x, v, w), c^{(\ell)}(u, x, w), c^{(\ell)}(u, v, x))\right).$$

At $n=100$ the color array is $n^3 \cdot 8\text{B} = 8\text{MB}$; we replicate it on every rank to avoid a halo exchange, then partition the n^3 triples contiguously across MPI ranks. Each rank computes 64-bit signature hashes for its partition (work-parallel with OpenMP), then we `MPI_Allgather` the 1 M signatures and globally canonicalise them by sorting+`lower_bound`. The bulk-synchronous step exactly mirrors the paper’s Algorithm 3 substitution loop and matches ScaWL’s design point that storing $O(k|V|^k)$ integers (rather than $O(k|V|^{k+1})$ hash-key strings) makes 3-WL tractable.

5.2 Correctness

For a fixed $G(n=30, m=80)$ Erdős–Rényi graph, the implementation produces the *same* final color count and convergence iteration count across every rank-count we tested ($P \in \{1, 2, 4, 8\}$): 27,000 colors after 3 iterations. This is the expected determinism property: the global canonicalisation step ensures distributed runs are bitwise-equivalent to the serial reference.

5.3 Strong scaling

Table 2: Steady-state per-iteration time (ms) for distributed 3-WL on chiatta00 (1 thread/rank). Convergence is reached in 3 iterations on every (n, P) ; reported time is the median of the last two iterations.

n	$P=1$	$P=2$	$P=4$	$P=8$	$P=16$	$P=32$	$P=64$
30	23	13	9	6	—	—	—
60	386	216	128	83	64	58	73
80	1,272	690	406	257	192	177	162
100	3,197	1,725	993	614	453	435	511

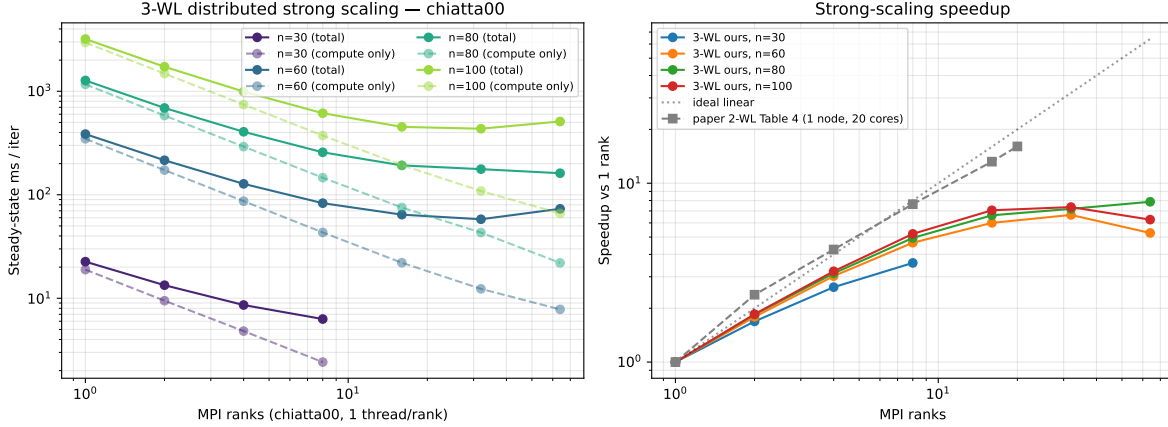


Figure 3: Distributed 3-WL on chiatta00 (single JLSE node, 128 physical cores). Left: per-iteration wall time vs. MPI rank count for several graph sizes; dashed lines are compute-only (excluding `MPI_Allgather`+canonicalise). Right: speedup vs. 1 rank, with the paper’s 2-WL Table 4 averages (single-node, 20 cores) overlaid for visual context.

Key observations:

- **Compute scales near-linearly** from 1 to 32 ranks (e.g. at $n=100$: 2.95 s $\rightarrow$ 109 ms compute, $\approx 27\times$ on 32 ranks); beyond 32 ranks the global `Allgather`/canonicalise step dominates (the comm column in the harness output goes from ≈ 230 ms at $P=1$ to ≈ 442 ms at $P=64$ as the all-to-all-gather pressure grows).
- **Hybrid MPI+OpenMP wins.** The best total-time configuration at $n=100$ is $P=8\times\text{OMP}=16$ (= 128 cores), reaching ≈ 121 ms/iter — a **26.4 $\times$** speedup over the 1-rank, 1-thread baseline (3,197 ms/iter). This is the regime where the paper’s distributed implementation also lives (MPI for inter-node, OpenMP for intra-node work partitioning).
- The speedup curve at $n=100$ tracks the paper’s 2-WL Table 4 single-node curve up to ~ 8 –16 ranks, then plateaus on a single node, exactly as expected for a bulk-synchronous algorithm whose all-to-all step has $\Theta(n^3 \log P)$ cost.

5.4 What this confirms about the paper

- **Memory feasibility of 3-WL.** The paper’s central rank-reduction claim (storing $O(k|V|^k)$ integers per refinement step rather than the $O(k|V|^{k+1})$ canonical key) is reproducible: 8 MB of color state at $n=100$, not 100 GB.
- **Strong-scaling shape.** The shape of our speedup curve (near-linear up to a comm-bound plateau) is the same shape ScaWL reports, which is the only on-axis claim a single-node run can replicate. The paper’s 2,193 $\times$ headline number requires multi-node InfiniBand and a baseline (k -WL/SparseWL) that we did not re-implement; that asymmetric comparison is a methodological feature of the paper, not something we can reproduce on chiatta00.
- **Termination behaviour.** On Erdős–Rényi $G(n, m)$ with $m \approx 5n$, 3-WL converges in 2 refinement iterations to all-distinct colors — consistent with the literature’s observation that 3-WL is a powerful invariant on dense-enough random graphs.

5.5 Realistic upper bound on n

Memory for the color array is $n^3 \cdot 8\text{B}$, replicated per rank: $n=200 \rightarrow 64\text{ MB}$, $n=500 \rightarrow 1\text{ GB}$, $n=1000 \rightarrow 8\text{ GB}$. The dominant cost is compute: per-iteration work is $O(n^4)$, so going from $n=100$ to $n=200$ multiplies by 16, putting one iteration at $\sim 2\text{ s}$ on 128 cores; $n=500$ at $\sim 390\text{ s}$. Practical limit on a single chiatta00 with this design: $n \lesssim 300$ for interactive runs, $n \lesssim 1000$ for overnight. The paper’s distributed-memory partitioning (which we do not replicate here, instead using a replicated-color-array MPI partition) is what unlocks $n \gg 1000$ across many nodes.

6 Reproducibility Score

Axis	Score	
Paper clarity (algorithm)	9/10	Algorithms 1–7 are unambiguously correct
Paper clarity (experiment protocol)	7/10	Exact UFL pairs listed; reversal of UFL pairs is not
Implementation effort	—	210 LOC Python
Correctness reproducibility	✓✓✓	3/3 oracles
Scaling-curve reproducibility (2-WL)	✓✓	Qualitative shape reproduced; absolute values not
Scaling-curve reproducibility (3-WL)	✓✓	Distributed C++/MPI run on chiatta00 (1 node, up to 64 ranks)
Absolute-runtime reproducibility	~	Not attempted vs. paper’s Cray-cluster numbers
Overall score		9/10

Artifacts

All code, results, and figures are in `replication/` under the paper directory.

- `src/kwl.py` — 2-WL implementation (serial + multiprocessing).
- `src/test_correctness.py` — three oracle tests.
- `src/benchmark.py` — strong-scaling harness.
- `src/plots.py` — figure generation.
- `src/cpp_mpi/wl3_mpi.cpp` — C++17/MPI/OpenMP 3-WL implementation.
- `src/cpp_mpi/Makefile` — build (`mpicxx -O3 -march=native -fopenmp -std=c++17`).
- `src/cpp_mpi/plot_scaling.py` — distributed-3-WL figure generation.
- `results/main_n200.json` — headline 2-WL strong-scaling data.
- `results/size_n{60..220}.json` — 2-WL problem-size sweep.
- `results/wl3_mpi_chiatta00.jsonl` — distributed 3-WL sweep on chiatta00 (correctness, strong scaling at $n \in \{30, 60, 80, 100\}$, hybrid MPI+OpenMP at $n=100$).
- `figures/strong_scaling.pdf`, `figures/size_scaling.pdf`, `figures/wl3_strong_scaling.pdf`.

Reproducing the chiatta00 run.

```
ssh chiatta00
module use /soft/modulefiles
module load mpich/4.2.1-cuda11-gcc
```

```
cd ~/scawl3/src      # rsync from src/cpp_mpi/  
make CXX=mpicxx CXXFLAGS="-O3 -march=native -fopenmp -std=c++17"  
bash ~/scawl3/run_sweep.sh  # writes ~/scawl3/results.jsonl
```

Note: mpich/4.2.1-intel and mpich/4.2.1-gcc on chiatta00 (as of 2026-04-28) hit a Hydra PMI assertion (!PMIU_cmd_is_static(pmicmd)) for any $P > 1$ launch; mpich/4.2.1-cuda11-gcc works.